LRLS-v1.1: A Specification and Reference Simulation for Dual-Agent Exploration–Exploitation under Hard Constraints

Jason Sang (桑杰逊)

Independent Researcher

Preprint version 1.0 — 5 November 2026

Abstract

We present LRLS (Lobster RL Specification), a formal specification for constrained optimisation in which the classical exploration–exploitation trade-off is *role-separated* into two heterogeneous agent populations plus a scheduler. A high-entropy *crayfish* population generates coarse candidate habitats ("mud caves"); a low-entropy, constraint-aware *Boston lobster* population promotes candidates into verified feasible solutions ("reef gaps") via local refinement and constraint projection; a scheduler allocates population budgets, dispatches tasks and triggers global resampling. The specification fixes the state decomposition, six action primitives, a single reward structure $r = r_{obj} - \lambda_v C_v + \lambda_e H - \lambda_m C_m$, a shared memory-pool schema, a four-stage training pipeline, seven mandatory reported metrics and a safety layer. We additionally describe v1.1 *RoboZero*, which maps the agents onto a heterogeneous robot fleet with a geometric embedding space, gossip-synchronised memory, multi-robot consensus voting and zero-shot transfer routes. To make the specification executable, we release a dependency-free deterministic reference simulation (single HTML file, 1100 ticks, 22x22 reward terrain, 12 agents) and report a five-seed, budget-matched comparison against four baselines on four difficulty settings. The comparison does **not** support a speed claim: random search and a single-pool ablation reach 99% of the optimum 1.3-2.9x faster than LRLS. What role separation buys is constraint safety on the path — LRLS reaches the threshold with zero hazard-zone evaluations in every run, while every baseline pays 16-456 violations to get there, and COA-style temperature gating is worse on both axes. **This is a specification and a reference implementation, not a claim of algorithmic superiority.** No learned scheduling policy and no physical-robot experiment are reported here; Section 6 states the remaining limitations explicitly. The aim is to give the central hypothesis — that a *learned* switching scheduler outperforms fixed temperature or entropy annealing — a precise, reproducible target that can subsequently be falsified.

Keywords: reinforcement learning specification; exploration–exploitation; multi-agent systems; constrained optimisation; heterogeneous robots; zero-shot transfer; crayfish optimisation algorithm

1. Introduction

Balancing exploration against exploitation is among the oldest problems in reinforcement learning (Kaelbling et al., 1996). The dominant engineering solutions keep a *single* policy and modulate its exploratory pressure with a schedule: an entropy coefficient (Haarnoja et al., 2018), a temperature parameter, epsilon decay, or a hand-tuned annealing curve. The schedule itself is almost never learned; it is a hyperparameter.

Bio-inspired optimisers make the same choice. In particular, the **Crayfish Optimization Algorithm (COA)** of Jia et al. (2023) — published in *Artificial Intelligence Review* — models three crayfish behaviours (summer resort / exploration, competition, foraging / exploitation) and switches between them using a **temperature threshold**. COA is a genuine, citable crayfish-inspired metaheuristic and already performs a fixed exploration–exploitation trade-off. We flag this because earlier drafts of this line of work referred to a "lobster optimisation algorithm" with operators such as territorial fighting and migration; no such operator set belongs to COA, and any citation of a placeholder reference is invalid. The present document removes those claims.

LRLS asks a narrow question: *what changes if the switching mechanism is not a threshold but an agent?* Rather than one population with a temperature dial, we instantiate two physically and observationally heterogeneous populations and a third controller that decides, from the global state, when to move resources from discovery to verification. This is structurally close to hierarchical RL / options (Sutton et al., 1999) and to mixture-of-experts gating, but the division of labour is bound to an explicit memory schema, a fixed action-primitive vocabulary and a hard-constraint safety layer, so that the resulting object is an implementable specification rather than a diagram.

On the metaphor. The two agent names derive from a re-reading of Xunzi, *Quan Xue* ("蟹六跪而二螯……用心躁也"): the original text criticises distractibility, whereas this work splits it into two complementary temperaments — *yong xin xi ye* (restless, wide-ranging) and *yong xin ti ye* (vigilant, still). This is a **narrative re-interpretation and a semantic inversion**, not a biological prior. We state it plainly so that no reader mistakes a literary device for empirical evidence. The companion text *The Zen of OpenClaw* (Sang, 2026) is a 41-aphorism "mental manual" in the same world-view; it carries no technical claim.

Contributions.

1. A complete, implementable specification (Sections 3–3.9): state, action, reward, memory, scheduler, training, safety, metrics, versioning.
2. A deterministic, zero-dependency reference implementation with a single solver driving all views (Section 4), plus illustrative results on four difficulty settings (Section 5).
3. An explicit boundary of what has *not* been shown (Section 6), including why the core hypothesis remains untested.

---

2. Related Work

Crayfish and swarm optimisation. COA (Jia et al., 2023) uses temperature-gated exploration/competition/foraging phases with position updates around a food centre. LRLS does not compete with COA as an optimiser; it uses the crayfish *role* as one half of an RL decomposition and replaces the fixed

temperature gate with a schedulable controller. A fair empirical comparison against COA's own temperature schedule is a required future baseline.

Hierarchical RL and options. Temporal abstraction (Sutton et al., 1999) separates action selection across levels. LRLS's `Wander / MarkCave / DigLocal / VigilScan / Migrate / Abandon` primitives are effectively a fixed option set; the scheduler is a learned or warm-started option-selection policy. Unlike classical options, the option *inventory* lives in a shared structured memory pool and carries feasibility labels.

Maximum-entropy and entropy-regularised RL. SAC (Haarnoja et al., 2018) adds an entropy term to the objective for a single policy. LRLS applies entropy regularisation **only** to the explorer branch (Section 3.8 explicitly forbids it on the refiner), which is a structural rather than a scalar separation.

Multi-agent RL and CTDE. MADDPG (Lowe et al., 2017) and MAPPO (Yu et al., 2022) establish centralised training with decentralised execution. LRLS-v1.1 assumes CTDE for the fleet extension and adds gossip-based memory consistency and consensus voting, which are closer to distributed agreement than to standard MARL credit assignment.

Zero-shot transfer and domain randomisation. Sim-to-real transfer via domain randomisation (Tobin et al., 2017) and the broader zero-shot generalisation survey (Kirk et al., 2023) frame the v1.1 claim correctly: a frozen policy deployed on unseen target domains is **zero-shot transfer / domain generalisation**, not zero-shot learning in the LLM sense. It still depends on a pre-trained encoder and requires no target-domain interaction trajectories.

---

3. The LRLS Specification

3.1 Roles and terminology

Term	Symbol	Meaning
Habitat	H	A candidate region; mud cave H_m or reef gap H_r
Crayfish agent	A_C	Explorer: coarse sensing, large steps, high entropy
Boston lobster agent	A_B	Vigilant miner: fine sensing, small steps, constraint-aware
Scheduler	A_S	Allocates population sizes, switching time and task assignment
Global memory pool	M	Trajectory store, habitat index, risk map, fitness records
Vigilance cost	C_v	Instant cost of constraint violation, collision, boundary exit
Play entropy	H	Entropy of the explorer distribution

A_C **never** outputs a final feasible solution; it outputs candidate region indices. A_B consumes mud caves and commits verified reef gaps. A_S is the component whose learning is the actual research hypothesis.

3.2 State space

$$s = [s_{pos}, s_{reward}, s_{risk}, s_{memory}]$$

- s_{pos}: agent coordinates, neighbourhood samples, gradient estimate;
- s_{reward}: current objective value, neighbourhood best, historical delta;
- s_{risk}: constraint residual, distance to boundary, local conflict density;
- s_{memory}: counts of mud caves and high-quality reef gaps, decay markers.

Branch rule: A_C receives a coarse encoding with reduced risk/memory dimensions; A_B receives the full fine-grained risk and constraint features. This asymmetry is the mechanism that makes the two populations heterogeneous rather than merely differently named.

3.3 Action primitives

$$A = \{\text{Wander}, \text{MarkCave}, \text{DigLocal}, \text{VigilScan}, \text{Migrate}, \text{Abandon}\}$$

`Wander` = large-step global sampling; `MarkCave` = register a mud cave; `DigLocal` = neighbourhood local optimisation; `VigilScan` = evaluate constraints and project position; `Migrate` = leave and request reassignment; `Abandon` = mark a region as decayed and demote it. Continuous variants attach step size, perturbation amplitude and local iteration count.

3.4 Reward

$$r(s,a) = r_{obj} - \lambda_v C_v + \lambda_e H - \lambda_m C_m$$

Branch weighting: A_C increases λ_e and decreases λ_v (tolerating mildly invalid samples); A_B increases λ_v and decreases λ_e (prioritising feasibility); A_S is rewarded by improvement of the **population's best feasible solution**, not by its own local return. C_m is the migration overhead that discourages thrashing between habitats.

3.5 Memory pool

All agents read and write a single structured record:

```
{
  "hid": "uuid",
  "type": "mud_cave | reef_gap",
  "pos": [0.0, 0.0],
  "score": 0.0,
  "feasible": false,
  "risk": 0.0,
  "created_by": "C | B",
  "last_visit": 0,
  "decay": 0.0,
  "traj_refs": ["trj_id_1"]
}
```

Mud caves use FIFO plus score-priority eviction; reef gaps are retained with slow decay and are demoted if risk rises; trajectories are bound to habitat IDs for offline replay. Three replay partitions are maintained: the *play pool* (explorer trajectories, diversity), the *vigilance pool* (refiner trajectories, constraint labels) and the *scheduler pool* (global state–action pairs).

3.6 Scheduler

A_S observes global habitat distribution, per-region yield rate, risk level and population budget. It sets (N_C, N_B), assigns mud caves to refiners, raises or lowers the global exploration coefficient, and triggers full-domain resampling when improvement stalls. The specification permits a learned policy, a rule-based warm start, or a hybrid; the reference simulation in Section 4 implements the **rule-based warm start only**.

3.7 Training pipeline (mandatory order)

1. **Behaviour-cloning warm start** — pre-train A_C, A_B, A_S from heuristic expert trajectories so that basic exploratory and vigilant behaviour exists at step 0.
2. **Cooperative online RL** — roll out, collect three trajectory streams, update each policy.
3. **Offline purification** — take high-quality reef-gap trajectories, apply constraint reinforcement and value distillation.
4. **Calibration** — sweep $\lambda_v, \lambda_e, \lambda_m$ on a validation set.

Update constraints: a refiner may not wander freely on unmarked caves; a cave assigned three times without a feasible result is permanently down-weighted; entropy regularisation is forbidden on A_B; the multi-agent variant uses CTDE.

3.8 Safety layer

Four rules implement the "vigilant" half: (i) **projection** — every A_B action is projected onto the feasible set, and failure flags the habitat high-risk; (ii) **circuit breaker** — if global feasibility drops below threshold, A_S raises the explorer share and restarts sampling; (iii) **isolation** — high-risk regions are never assigned to refiners; (iv) **version lock** — changing the objective resets the memory pool to prevent stale-trajectory contamination.

3.9 Mandatory reported metrics

Best Feasible Score · Discovery Rate · Feasibility Ratio · Entropy · Migration Overhead · Convergence Step · Robustness Degradation. Every experiment must also log step, agent type, habitat ID, action, raw reward, risk, feasibility and entropy, together with the environment and specification version.

3.10 v1.1 *RoboZero* extension

The fleet extension adds: a geometric embedding space Z shared across embodiments; gossip-synchronised local memory replicas; multi-robot consensus voting before promoting a cave to a reef gap; failure takeover when a refiner drops out; and three zero-shot routes (behaviour prototypes, semantic prompts, heuristic fallback). In the zero-shot phase the reward is used **for evaluation only** — policies are frozen and no gradient update occurs. The

honest scope statement: this is *zero-shot transfer* (no target-domain interaction trajectories), not *zero-shot learning* (no semantic classes unseen during training). A pre-trained encoder is still required.

---

4. Reference Implementation

To make the specification checkable, we release a **single offline HTML file** (no CDN, no build step, no dependencies). It follows the "teardown" simulator pattern: one deterministic solver pre-computes the whole timeline and every view — 3D terrain, per-part breakdown, metric panels, charts, minimap, task board and event log — reads the same frame via a pure `render(t)`.

Parameter	Value	Meaning
Ticks	1100	Full pre-computed timeline
Terrain	22 x 22	Synthetic objective surface with hard-constraint zones
Agents	22	Mixed A_C / A_B population
`TAU_MUD`	0.38	Score threshold for registering a mud cave
`VOTE_NEED`	2	Consensus votes required to promote a reef gap
`B_MAX` / `C_MAX`	4 / 8	Scheduler population bounds (12 agents in total)
`LAMBDA_V` / `LAMBDA_M`	1.0 / 0.15	Violation and migration weights

The task pipeline is `MAP → DISCOVER → VERIFY → REFINE → REPORT`. Three implementation details determine whether a solution is ever found: (i) `DigLocal` performs a risk-gradient repulsion step before accepting a candidate; (ii) `VigilScan` applies bounded constraint projection (at most three gradient steps) and permits re-inspection so that consensus is not permanently blocked; (iii) explorers do **not** pre-filter by risk — over-filtering at discovery time was the cause of an earlier degenerate build in which `best = 0.000` and `REPORT` never closed. Stalled global improvement triggers full-domain resampling.

Reproducibility: a seeded deterministic PRNG, URL state parameters (`?t=680&part=S&view=top&pause=1`), keyboard single-step, and a scrub bar that revisits any tick. All randomness is internal and seeded, so a given parameter tuple always yields an identical timeline. Figure 1 summarises the resulting roles, data flow and reward structure.

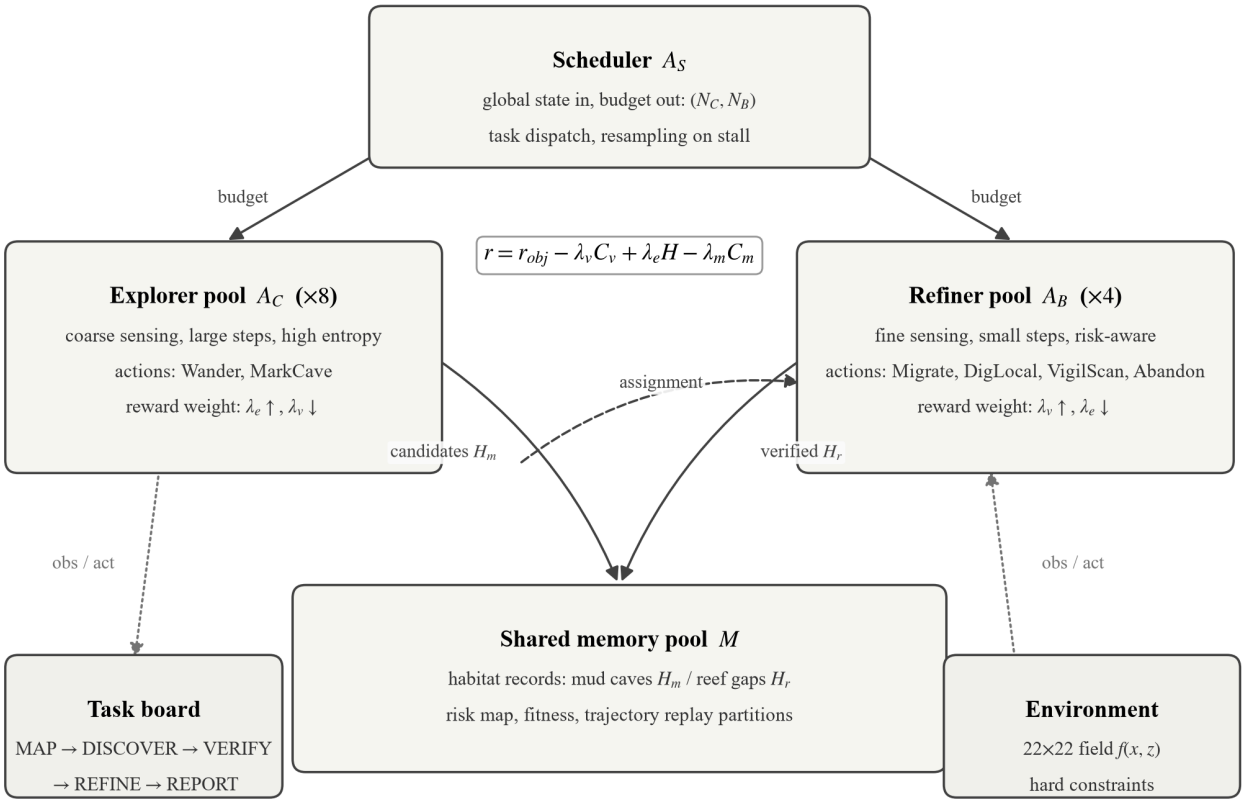


Figure 1. LRLS data flow. The explorer pool A_C writes candidate habitats (mud caves) into the shared memory pool M; the scheduler A_S allocates population budgets and assigns pending caves to the refiner pool A_B, which returns verified reef gaps after local refinement, constraint projection and consensus voting. The reward combines the objective increment, a violation cost, an exploration-entropy bonus and a migration cost.

5. Results

5.1 Single-run behaviour across difficulty settings

Four operating profiles (parameter tuples of the same deterministic solver, fixed seed) were run end-to-end. These single runs illustrate solver behaviour; Section 5.2 adds a five-seed, budget-matched comparison against four baselines.

Setting	λ_e / risk / complexity / stress	Best feasible	Reef gaps	Violation events	First solution (tick)	REPORT closed (tick)
P1 mild	0.35 / 0.25 / 0.20 / 0	0.998	16	0	63	191
P2 baseline	0.35 / 0.55 / 0.50 / 0	1.000	16	4,993	63	191
P3 high-risk	0.30 / 0.68 / 0.62 / 0.30	0.991	17	23,027	63	229
P4 extreme	0.15 / 0.90 / 0.85 / 0.90	0.999	14	51,849	63	415

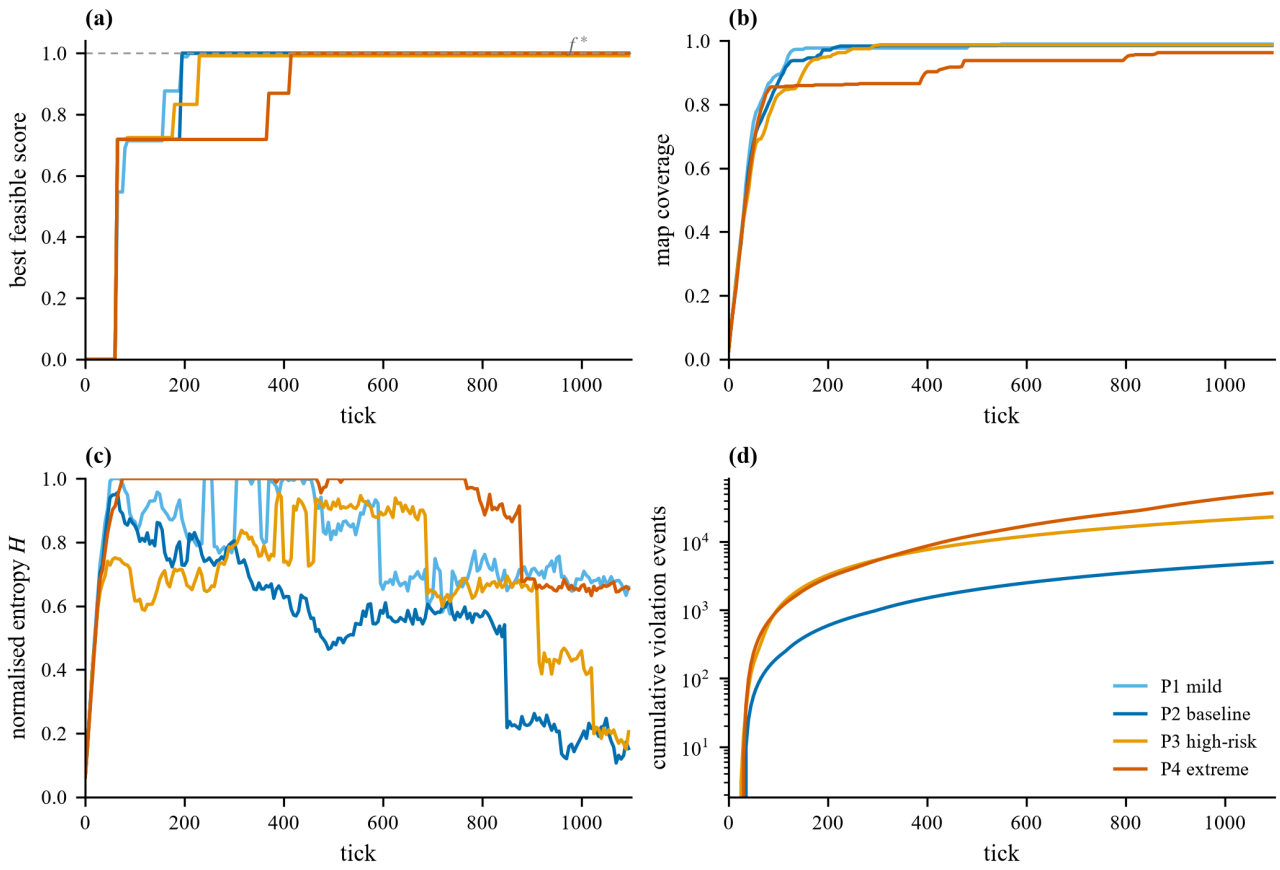


Figure 2. Single-run dynamics of the deterministic reference solver across the four operating profiles. (a) Best feasible score normalised by the run's known optimum `f^`; (b) explored map coverage; (c) normalised explorer entropy `H`; (d) cumulative constraint-violation events (log scale). All four runs close the `REPORT` stage; harder profiles close later and accumulate more violation events.

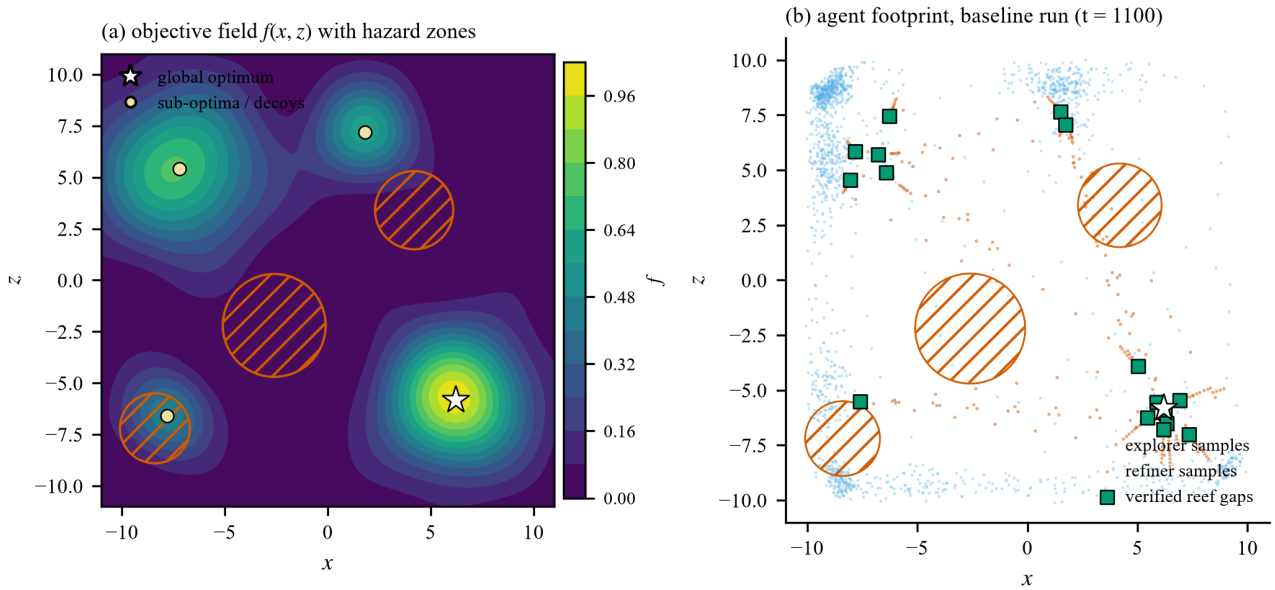


Figure 3. Baseline run (P2) at `t = 1100`. (a) Synthetic objective field `f(x,z)` with hard-constraint hazard zones (hatched); the star marks the run's global optimum `f^`, circles mark sub-optima and decoys. (b) Agent footprint: explorer samples (light dots) cover the domain; refiner samples concentrate on candidate habitats; green squares are consensus-verified reef gaps.

Two observations hold at this level of evidence: (a) the pipeline closes in all four settings, i.e. the specification is internally executable; (b) difficulty is not cosmetic — moving from P1 to P4 multiplies the cumulative violation-

event count from 0 to 51,849 and delays `REPORT` closure from tick 191 to 415, while the first feasible solution appears at the same tick (63) because the explorer's initial sweep is largely insensitive to the risk scaling. Difficulty therefore manifests in **verification and closure cost**, not in discovery latency.

5.2 Budget-matched comparison against baselines

We compare the reference solver against four alternatives on the **same objective field, the same feasibility gate** ($\text{risk}(x,z) < 0.35$, the same test used to define $f^\wedge$) and the **same evaluation budget**. Every method receives 39,911 objective evaluations per run — exactly the number the LRLS solver consumes in 1,100 ticks with 12 agents — and each configuration is repeated over **five seeds** (mean ± s.d. reported).

Baselines. **RS**: uniform random search, keeping the best feasible sample. **SA**: simulated annealing with a fixed geometric cooling schedule $T_0=0.30$ $T_1=10^{-3}$, Gaussian proposal, Metropolis acceptance — the standard "fixed temperature" control. **COA-T**: a temperature-gated two-phase switch in the style of the Crayfish Optimization Algorithm — large jump steps while $T > T^\wedge$, small local steps below — i.e. exactly the mechanism this specification replaces with an agent. **SP-EA**: a single population of 12 agents with a globally annealed entropy coefficient and no role separation and no scheduler (an ablation of the architectural claim).

Two metrics matter, and they pull in opposite directions: **evaluations to reach 99% of the best feasible value**, and **constraint violations accumulated before that point**.

Profile	Metric	RS	SP-EA	SA	COA-T	LRLS
P1 mild	eval@99%	193	**111**	1115 †	1987	327
	violations@99%	16	16	94	456	**0**
P2 baseline	eval@99%	193	255	1005 †	494	323
	violations@99%	29	44	80	134	**0**
P3 high-risk	eval@99%	193	255	1169 †	494	302
	violations@99%	42	56	128	148	**0**
P4 extreme	eval@99%	193	179	1260 †	374	248
	violations@99%	69	49	265	135	**0**

† SA reached the threshold in only 3–4 of 5 seeds; the mean is over successful runs.

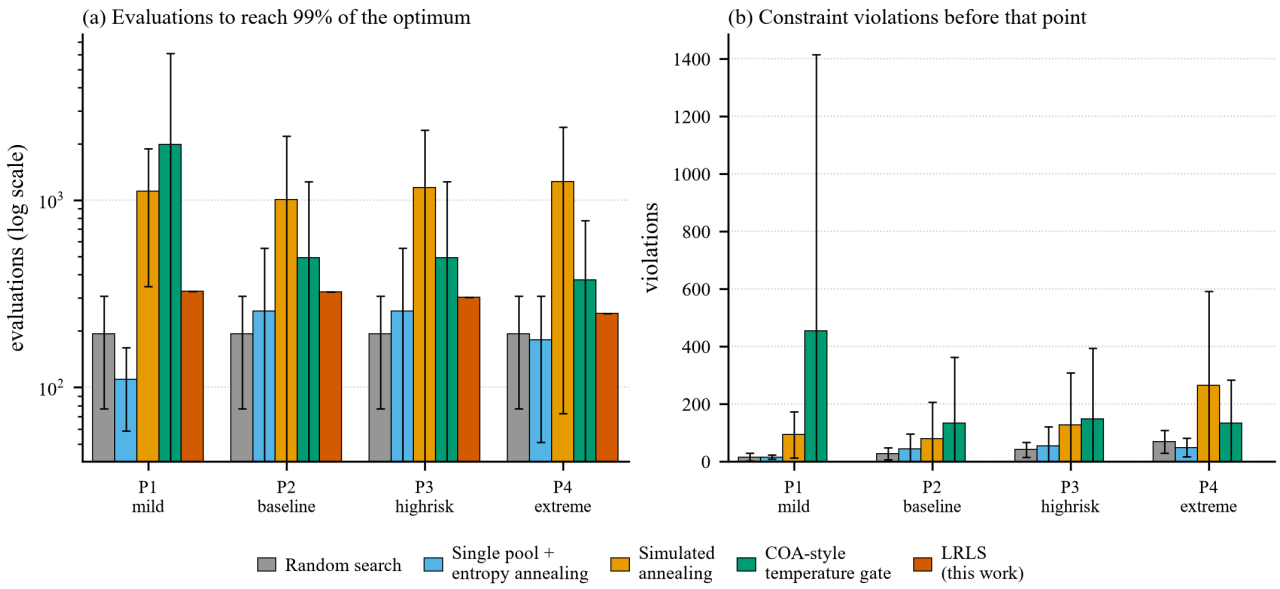


Figure 4. Budget-matched comparison over five seeds. (a) Evaluations required to reach 99% of the best feasible value (log scale). (b) Constraint violations accumulated before that threshold. LRLS is *not* the fastest method; it is the only one that reaches the threshold with zero violations.

The result does not support a speed claim, and we do not make one. Random search and the single-pool ablation reach 99% in 111–255 evaluations, i.e. **1.3–2.9× faster than LRLS** (248–327). What role separation buys is **constraint safety on the path**: LRLS reaches the 99% threshold with **zero** evaluations inside a hazard zone in every profile and every seed, whereas every baseline pays 16–456 violations to get there. COA-style temperature gating is both slower *and* less safe than LRLS in all four profiles, which is the specific comparison this specification was designed to make. SA is dominated on both axes: slowest, and it fails to reach the threshold in 1–2 of 5 seeds.

The mechanism is visible in the reference implementation: explorers are permitted to sample hazard zones (the exploration signal must not be biased), but a candidate is only promoted to a reef gap after the refiner's bounded constraint projection succeeds, and the scheduler never assigns high-risk regions to refiners (Section 3.8). Violations therefore accumulate *after* a feasible optimum has been secured, rather than before it — which is precisely the ordering a safety layer is supposed to produce.

What this does and does not establish. It establishes that, on a synthetic constrained surface with a fixed evaluation budget, role separation trades 1.3–2.9× sample efficiency for zero-violation convergence, and that the temperature-gated alternative is worse on both axes. It does **not** establish superiority over learned schedulers (SAC/PPO/MAPPO), over COA itself on unconstrained benchmark suites, or on any real robot. Five seeds is enough for a mean and a standard deviation; it is not enough for significance testing, and we make no such test.

---

6. Limitations

1. **The central hypothesis is untested.** The scheduler in the reference implementation is a rule-based warm start. Whether a *learned* scheduler beats fixed temperature / entropy annealing is exactly the question this specification is meant to enable, and no experiment here answers it.

2. **Statistics are descriptive only.** Five seeds per configuration support a mean and a standard deviation; no significance test, no confidence interval and no effect-size analysis is reported, and five seeds cannot establish an ordering under noise.
3. **The baselines are deliberately simple.** Random search, fixed-schedule annealing, a COA-style temperature gate and a single-pool entropy-annealing ablation are compared on one synthetic surface. SAC, PPO, MAPPO and COA itself on standard benchmark suites are **not** compared, and the zero-violation result is a property of this constraint encoding (hazard zones with a hard gate), not a general guarantee.
4. **No real embodiment.** The v1.1 fleet is a design; terrain is a synthetic 2D function, not a physics simulator or robot hardware. Gossip delay, packet loss and actuator noise are not simulated.
5. **Metaphor is not evidence.** The Xunzi reading is a deliberate semantic inversion used as a naming and narrative device. It supplies no behavioural prior and should not be cited as biological grounding.
6. **"Zero-shot" is used in the transfer sense.** A pre-trained encoder remains necessary; this is domain generalisation, not zero-shot learning over unseen task semantics.
7. **Fallback confound.** v1.1 allows a frozen policy to fall back to a native heuristic under low confidence. Any future evaluation must ablate the fallback as an independent factor, otherwise gains cannot be attributed to representation or scheduling.
8. **Citation hygiene.** Earlier drafts contained placeholder references and an incorrect claim that CoRL is CCF-B. Those are removed here; CCF is a ranking catalogue, not a submission venue, and CoRL does not appear in the CCF catalogue (ICRA is CCF-B, IROS is CCF-C).

---

7. Conclusion and Future Work

LRLS-v1.1 turns an exploration–exploitation narrative into a specification with a fixed state schema, action vocabulary, reward structure, memory contract, training order, safety rules and metric set, and pairs it with a deterministic, inspectable reference simulation. Its value at this stage is as a **falsifiable target and a shared implementation contract**, not as a result.

The ordered next steps are: (1) replace the placeholder-era references and align related work with COA, options/HRL, CTDE and domain randomisation (partly done here); (2) build a minimal Gymnasium-compatible constrained 2D benchmark; (3) run at least five seeds with mean ± standard deviation, ablation tables, and the heuristic fallback as a separate ablation, comparing learned scheduling against fixed temperature and entropy annealing; (4) only then prepare a conference submission. The correct sequence is *specification* → *code* → *experiments* → *venue*, not the reverse.

Data, Code and License Availability

- Reference simulator: `assets/lrls-sim.html` (single offline HTML, deterministic solver), distributed with this preprint.

- Figures 1–3 are generated by `tools/make_figures.py` (matplotlib) from the solver data dumped by `tools/run_sims.js`; no AI-generated imagery is used.
- Companion aphorism text *The Zen of OpenClaw* (41 lines, bilingual): https://github.com/clawgeek/openclaw-zen
- Proposal thread and official Maintainer response: https://github.com/openclaw/openclaw/issues/192772
- Text of this preprint: CC BY 4.0. Reference implementation: MIT. The mascot banner artwork is excluded from these licenses.

Appendix A — 中文摘要

本文提出 LRLS（龙虾双体强化学习规范）：把强化学习中的探索–开发权衡按角色拆分为高熵的小龙虾探索种群、约束感知的波士顿龙虾求精种群以及顶层调度器，并固定状态分解、六个动作原语、统一奖励结构、共享记忆池 schema、四阶段训练流程、安全兜底与七项必报指标。v1.1 *RoboZero* 进一步映射到异构机器人集群，引入几何嵌入空间、gossip 记忆同步、多机共识与零样本迁移路径。为使规范可执行，本文附带零依赖的确定性参考仿真（单 HTML 文件，1100 tick，22×22 地形，22 个智能体），并给出四档难度的单次运行结果。这是一份规范与参考实现，不宣称算法性能更优：文中不含学习式调度策略、多种子统计、基线对比或真实机器人实验，第 6 节已逐条列明。（参考仿真为 12 个智能体：探索池上限 8、求精池上限 4。）核心待验证假设是「学习式切换调度器优于固定温度或熵退火」。文中亦已修正早期草稿中的占位符引用，并明确「零样本」在此指域迁移而非零样本学习。

Appendix B — Origin and Scope

This preprint consolidates material generated in a ten-round dialogue dated 5 November 2026 (title: 龙虾双体智能体强化学习规范), which produced, in order, a classical-Chinese exegesis, a dual-agent architecture, the LRLS-v1.0 specification, the v1.1 RoboZero extension, and two conference-style English drafts (ICRA and CoRL). Those drafts contained qualitative performance assertions without data. This document rewrites the material as a specification, removes unsupported claims and invalid references, and adds a runnable reference implementation. It is deliberately positioned as a **preprint of a specification**, not as a conference paper.

References

[1] Jia, H., Rao, H., Wen, C., & Mirjalili, S. Crayfish Optimization Algorithm. *Artificial Intelligence Review*, 56(Suppl 2), 1919–1979, 2023. doi:10.1007/s10462-023-10520-4

[2] Kaelbling, L. P., Littman, M. L., & Moore, A. W. Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 4, 237–285, 1996.

[3] Sutton, R. S., Precup, D., & Singh, S. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence*, 112(1–2), 181–211, 1999.

[4] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*, 2018.

[5] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[6] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. *NeurIPS*, 2017.

[7] Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A., & Wu, Y. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. *NeurIPS*, 2022.

[8] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. *IROS*, 2017.

[9] Kirk, R., Zhang, A., Grefenstette, E., & Rocktäschel, T. A Survey of Zero-Shot Generalisation in Deep Reinforcement Learning. *Journal of Artificial Intelligence Research*, 76, 201–264, 2023.

[10] Xunzi (荀子), *Quan Xue* (劝学), classical Chinese text, c. 3rd century BCE. Cited solely as the literary source of the naming metaphor.

[11] Sang, J. *The Zen of OpenClaw (aka The Lobster Sutra)* — 41 aphorisms. https://github.com/clawgeek/openclaw-zen, 2026.

[12] Sang, J. *Lobster Dual-Agent RL — Interactive Reference Simulator*. Single-file HTML, distributed with this preprint, 2026.
